

Jev vs LLM

Designing a safe refund workflow

A focused student reference based on the video scenario

Scenario

A customer writes: **“You charged me twice. Refund the extra payment.”**

The message tells us what the customer **claims**. It does not prove that two settled charges exist, that the second charge is refundable, or that a refund has not already been issued.

CLAIM $\neq$ VERIFIED FACT

Core rule: Model output can describe, classify, score, or route a request. Money movement must be controlled by verified records and deterministic policy.

Safe architecture in one line

JEV ROUTES



CODE VERIFIES + ACTS



LLM EXPLAINS

Student objective

Know exactly where probabilistic prediction stops and application authority begins.

Jev owns

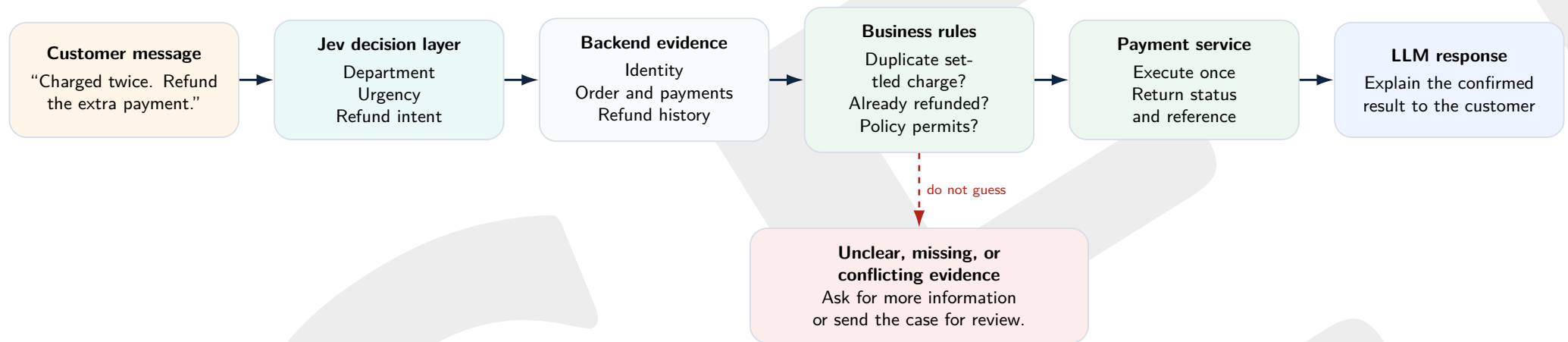
Focused decisions such as department, urgency, and refund intent, returned as constrained values with probabilities.

Backend owns

Identity checks, transaction lookup, policy evaluation, idempotency, authorization, execution, and audit logs.

LLM owns

A clear customer explanation based only on the confirmed status, amount, timing, and refund reference.



MESSAGE → JEV → DEPARTMENT · URGENCY · INTENT
IDENTITY → ORDER → DUPLICATE CHARGE → REFUND HISTORY
PAYMENT CONFIRMS → STATUS + REFERENCE → LLM EXPLANATION

What Jev contributes

It turns shared text or structured state into typed, focused judgments with probabilities. That output can help software choose a route or decide when to escalate.

What actually authorizes the refund

Verified payment records plus deterministic business rules. The model does not hold the payment credentials or bypass policy checks.

Typical generative LLM

TOKEN → TOKEN → TOKEN

A generative LLM normally produces text autoregressively. It can also be constrained to JSON, but the application still needs schema validation and must prevent generated language from becoming authorization.

Jev focused decisions

Q1 || Q2 || Q3

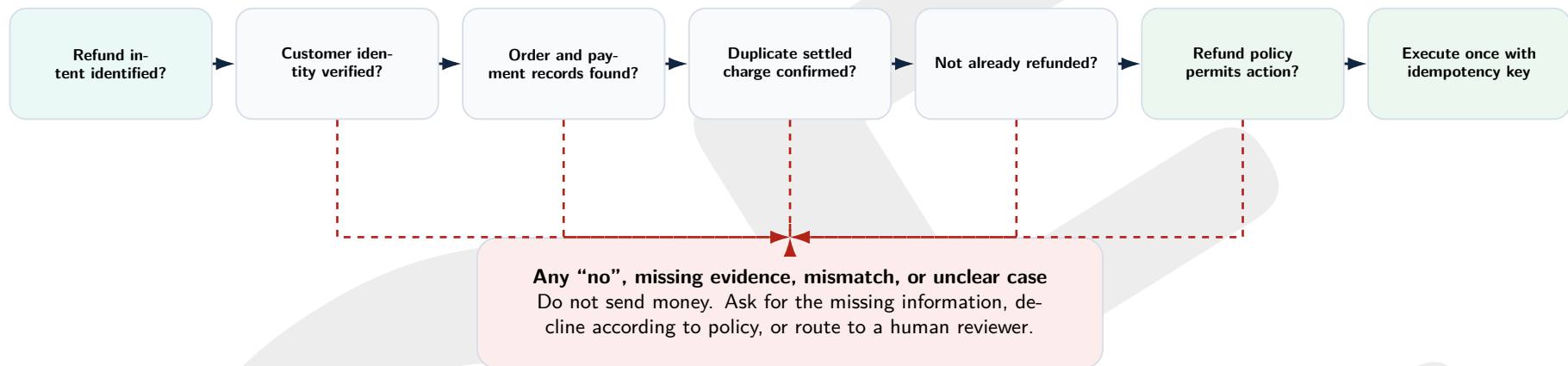
TypeSafe describes Jev as evaluating declared questions against the same state and returning constrained answers plus probabilities. Independent questions can be evaluated together.

Dimension	Generative LLM	Jev / System One model
Primary output	Generated text or structured text	Typed choices, scores, and boolean-style probabilities
Useful here for	Explaining a confirmed outcome in natural language	Classifying, routing, scoring, and identifying intent
Execution pattern	Output tokens are normally generated sequentially	Declared independent questions are evaluated together
Critical boundary	JSON does not itself authorize a refund	Probability does not prove transaction evidence
Selection rule	Measure quality, latency, cost, reliability, and operational fit	Measure the same factors on the same production-like test set

Benchmarking note. TypeSafe publishes substantial speed and cost claims for Jev on its workflow evaluations. Treat those as provider-reported results, not a universal guarantee. Benchmark your own inputs, network region, question set, thresholds, retry policy, and error costs.

Refund authorization flow

CLASSIFY FIRST, AUTHORIZE WITH EVIDENCE



Confidence is not evidence

A high model probability cannot replace a missing ledger entry, settlement status, or refund record.

Idempotency is mandatory

Use a stable operation key so retries cannot issue the same refund twice.

Explain only after confirmation

The LLM receives the final status and reference from the backend, not a guessed outcome.

JEV ROUTES

RULES AUTHORIZE

PAYMENT CONFIRMS

Illustrative focused output

Example shape only. This is not a live Jev response.

```
{
  "department": {
    "value": "billing",
    "probability": 0.97
  },
  "urgency": {
    "score": 1.4
  },
  "refund_requested": {
    "value": true,
    "probability": 0.95
  }
}
```

Deterministic authorization logic

The application, not the model, owns this gate.

```
if not identity_verified:
    return REVIEW

payment = lookup_order_and_payments()

eligible = (
    payment.duplicate_charge
    and payment.settled
    and not payment.already_refunded
    and policy_allows_refund(payment)
)

if not eligible:
    return REVIEW_OR_DECLINE

return refund_once(
    payment_id=payment.id,
    idempotency_key=case_id
)
```

Confirmed facts → LLM → customer explanation

Backend facts: refund status = successful; reference = RF-48219; amount = confirmed amount; expected settlement window = confirmed window.

Safe response: “The duplicate payment has been refunded. Your refund reference is RF-48219. The amount and expected timing shown here come from the confirmed transaction record.”

Production evaluation checklist

DECISION ACCURACY

CALIBRATION

LATENCY

COST

FALSE-ACTION RATE

ESCALATION QUALITY

Test with real ticket language, ambiguous cases, missing records, conflicting records, retries, outages, and already-refunded transactions.

Official references

[TypeSafe AI: System One](#)

[TypeSafe AI: Introducing Jev](#)

[Vercel: classify, route, and score with Jev](#)

Checked 20 September 2026. Educational reference; not a TypeSafe publication or a live model test.

JEV ROUTES

· CODE VERIFIES + ACTS

· LLM EXPLAINS

Choose every component using measured accuracy, latency, cost, and operational risk.